

1. What is an Array?

- An **Array** in Java is a **data structure** used to store multiple values of the **same data type** in a **contiguous memory location**.
- It is a **fixed-size** collection, meaning once an array is created, its size cannot be changed.
- It is an **indexed collection**: each element can be accessed using an index (starting from 0).

👉 Example: Storing 5 integers in an array instead of creating 5 separate variables.

2. Key Features of Arrays

1. **Homogeneous data** – stores same type of elements (int, float, String, etc.).
2. **Index-based** – first element at index 0, last element at index length-1.
3. **Fixed size** – size must be defined at creation.
4. **Stored in contiguous memory** for fast access.
5. **Default values** –
 - Numeric → 0
 - Boolean → false
 - Objects → null

3. Array Declaration & Initialization

(i) Declaration

```
// Declaration  
int[] arr;    // preferred  
int arr[];    // valid but less used
```

(ii) Memory Allocation

```
// Allocation  
arr = new int[5]; // size = 5
```

(iii) Declaration + Allocatio

```
int[] arr = new int[5];
```

(iv) Initialization

```
int[] arr = {10, 20, 30, 40, 50}; // direct initialization
```

◆ 4. Accessing Array Elements

```
int[] arr = {10, 20, 30, 40, 50};  
  
System.out.println(arr[0]); // 10  
System.out.println(arr[4]); // 50  
System.out.println(arr.length); // 5 (size of array)
```

6. Traversing an Array

(i) Using for loop

```
for(int i = 0; i < arr.length; i++) {  
    System.out.println(arr[i]);  
}
```

(ii) Using enhanced for loop

```
for(int num : arr) {  
    System.out.println(num);  
}
```

7. Types of Arrays

1. One-dimensional array

```
int[] arr = {1, 2, 3, 4, 5};
```

2. Multi-dimensional array (Matrix)

```
int[][] matrix = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};
```

Access:

```
System.out.println(matrix[1][2]); // 6
```

8. Difference Between Array and Array List

Feature	Array	Array List
Size	Fixed	Dynamic (can grow/shrink)
Data Type	Can store primitives & objects	Stores only objects
Performance	Faster	Slightly slower (due to resizing)
Utility Methods	Limited	Rich set of methods (add, remove, contains, etc.)

1. Advantages of Arrays

1. **Code Simplification** – Instead of multiple variables, one array can store all values.

```
int a1=10, a2=20, a3=30; // messy
int[] arr = {10, 20, 30}; // clean
```

2. **Fast Access** – Elements are accessed using **index** in $O(1)$ time.
3. **Memory Efficiency** – Stored in **contiguous memory** → improves CPU caching.

4. **Ease of Traversal** – Looping and enhanced for loop makes iteration easy.
5. **Homogeneous Storage** – Stores elements of **same data type** together.
6. **Default Initialization** – Arrays automatically initialize with default values (0, false, null).

◆ 2. Disadvantages of Arrays

1. **Fixed Size** – Cannot grow/shrink dynamically. Once declared, size is constant.
2. **Wasted Memory** – If size is large and elements are fewer → unused space.
3. **Insertion/Deletion is Expensive** – Shifting elements required.
Example: Inserting at beginning → shift all elements right.
4. **Stores only Homogeneous Data** – Cannot mix data types in one array.
5. **No Built-in Methods** – No direct support for add/remove/contains (need ArrayList for dynamic operations).

Explanation of Indexing in Arrays (Java)

1. What is Indexing?

- An **array** stores multiple elements in a **linear sequence**.
- Each element is placed at a **position number**, called an **index**.
- In **Java**, indexing always starts from **0**.
- So:
 - The **first element** → index 0
 - The **second element** → index 1
 - ...
 - The **last element** → index length - 1

2. Example

```
int[] arr = {10, 20, 30, 40, 50};
```

Here is how indexing works:

Index	Element
0	10
1	20
2	30
3	40
4	50

👉 arr[0] = 10

👉 arr[2] = 30

👉 arr[4] = 50

3. Key Points about Indexing

1. **Zero-based** → First element at index 0.
2. **Range** → Valid indexes are from 0 to length - 1.
3. **Direct Access** → Any element can be accessed in **O(1) time** using its index.
4. **Out of Range Error** → If you try to access an index outside the range, Java throws **ArrayIndexOutOfBoundsException**.

4. Example Program

```
public class ArrayIndexing {  
    public static void main(String[] args) {  
        int[] arr = {10, 20, 30, 40, 50};  
  
        // Accessing elements using index  
        System.out.println("First element: " + arr[0]);  
        System.out.println("Third element: " + arr[2]);  
        System.out.println("Last element: " + arr[arr.length - 1]);  
  
        // Invalid index  
        // System.out.println(arr[5]); // ✖ Throws  
        // ArrayIndexOutOfBoundsException  
    }  
}
```

5. Indexing in Multi-Dimensional Arrays

In a 2D array, indexing is done using **row index** and **column index**:

```
int[][] matrix = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

```
System.out.println(matrix[0][0]); // 1
```

```
System.out.println(matrix[1][2]); // 6
```

👉 `matrix[1][2]` means → **row 1, column 2** (value = 6).

CODE:-

```
java Copy code  
  
package BasicConcept; // Package name (folder where class is stored)  
  
public class arraydemo { // Class definition  
  
    public static void main(String[] args) { // Main method → program entry point  
  
        // Declare and initialize an integer array with 5 elements  
        int ar[] = {10,20,30,40,50};  
  
        // Print the length (size) of the array using the .length property  
        System.out.println("length of array:" + ar.length);  
  
        // For loop to traverse array (runs from index 0 to ar.length-1)  
        for(int i = 0; i < ar.length; i++) {  
            // currently empty → means no operation is performed inside the loop  
            // (you could print or process each element here)  
        }  
    }  
}
```

OUTPUT : length of array:5

Step-by-step Explanation

- package BasicConcept; → Defines the package (folder) where this class is stored.
- public class arraydemo → Defines a public class named arraydemo.
- public static void main(String[] args) → Entry point of the program.
- int ar[] = {10,20,30,40,50}; → Creates an integer array with 5 elements.

- `System.out.println("length of array:" + ar.length);` → Prints the array length (5).
- `for(int i=0;i<ar.length;i++) { }` → A loop that goes through each element (0 to 4), but it currently does nothing inside.

Even odd code

```
package assignment; // Defines the package (folder name)

public class evennumber { // Class definition

    public static void main(String[] args) { // Main method → program starts here

        // Declare and initialize an integer array
        int [] arr = {10, 20, 23, 25, 26, 30};

        // Print heading message
        System.out.println("even number in array");

        // Loop through all elements of the array
        for(int i = 0; i < arr.length; i++) {

            // Check if the element is divisible by 2 (even number condition)
            if(arr[i] % 2 == 0) {

                // Print the even number
                System.out.println(arr[i]);
            }
        }
    }
}
```

Step-by-step Explanation

1. `package assignment;` → Puts this class inside the assignment package (folder).
2. `public class evennumber` → Declares the class named evennumber.

3. `int [] arr = {10,20,23,25,26,30};` → Creates an integer array with 6 numbers.
 4. `System.out.println("even number in array");` → Prints the title message.
 5. `for(int i=0; i<arr.length; i++)` → A **for loop** to go through each element of the array.
 6. `if(arr[i] % 2 == 0)` → Checks if the current element is **even** (divisible by 2).
 7. `System.out.println(arr[i]);` → Prints the number if it is even.
-

◆ Output

even number in array

10

20

26

30

.

```
package BasicConcept; // Package name (folder)

// Import Scanner class for taking user input
import java.util.Scanner;

public class arraydemo2 { // Class definition

    public static void main(String[] args) {

        // Create Scanner object to read input
        Scanner sc = new Scanner(System.in);

        // Ask user for the number of cities
        System.out.println("Enter limit: ");
        int n = sc.nextInt(); // Store number of cities

        // Declare a String array of size 'n'
        String city1[] = new String[n];

        // Ask user to enter city names
        System.out.println("Enter city names:");
        for(int i = 0; i < n; i++) {
            city1[i] = sc.next(); // Read city name and store in array
        }
    }
}
```

 Copy code

```
// Create Scanner object to read input
Scanner sc = new Scanner(System.in);

// Ask user for the number of cities
System.out.println("Enter limit: ");
int n = sc.nextInt(); // Store number of cities

// Declare a String array of size 'n'
String city1[] = new String[n];

// Ask user to enter city names
System.out.println("Enter city names:");
for(int i = 0; i < n; i++) {
    city1[i] = sc.next(); // Read city name and store in array
}

// Display all entered city names
System.out.println("City names are");
for(int i = 0; i < n; i++) {
    System.out.println(city1[i]); // Print each city
}
}
```

 Copy code

◆ Step-by-step Explanation

1. `import java.util.Scanner;` → Allows us to take user input from the keyboard.
 2. `Scanner sc = new Scanner(System.in);` → Creates a scanner object.
 3. `int n = sc.nextInt();` → Reads the **number of cities** the user wants to enter.
 4. `String city1[] = new String[n];` → Declares an array of Strings to hold city names.
 5. `for(int i=0; i<n; i++) { city1[i] = sc.next(); }` → Reads city names one by one and stores them in the array.
 6. Another **for loop** prints all stored city names.
-

◆ Example Execution

Input:

Enter limit:

3

Enter city names:

Pune

Mumbai

Delhi

Output:

City names are

Pune

Mumbai

Delhi

1. One-Dimensional (1D) Array

A **1D array** is simply a list of elements stored in contiguous memory locations.

Syntax

java

 Copy code

```
datatype arrayName[] = new datatype[size];
```

java

 Copy code

```
public class OneDArray {
    public static void main(String[] args) {
        int[] arr = new int[5]; // Declare 1D array of size 5

        // Initializing values
        arr[0] = 10;
        arr[1] = 20;
        arr[2] = 30;
        arr[3] = 40;
        arr[4] = 50;

        // Printing values
        System.out.println("1D Array Elements:");
        for(int i=0; i<arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

✓ Output:

mathematica

 Copy code

1D Array Elements:

10 20 30 40 50

2.Two-Dimensional (2D) Array

A **2D array** is like a **matrix** (rows and columns).

SYNTAX-

java

 Copy code

```
datatype[][] arrayName = new datatype[rows][cols];
```

Line-by-Line Explanation

package BasicConcept;

- Defines the package name as BasicConcept.
- Packages in Java are used to group related classes together.
- Here, your class array2d belongs to the BasicConcept package.

public class array2d {

- Declares a **public class** named array2d.
- public → accessible from anywhere.
- class → blueprint for objects, but here we just use it to hold our main method.

java

 Copy code

```
package BasicConcept;  
  
public class array2d {  
  
    public static void main(String[] args) {  
        int arr[][]= { {10,20},{30,40} };  
  
        for( int r=0;r<arr.length;r++) {  
            for( int c=0;c<arr.length;c++) {  
                System.out.print(arr[r][c]+" ");  
            }  
            System.out.println();  
        }  
  
    }  
  
}
```


public static void main(String[] args) {

- Entry point of the Java program.
 - public → JVM can access it from anywhere.
 - static → method belongs to the class, not objects.
 - void → returns nothing.
 - String[] args → command-line arguments (not used here).
-

int arr[][] = { {10,20}, {30,40} };

- Declares and initializes a **2D array**.
- Structure of arr:
-

mathematica

 Copy code

Row 0 → {10, 20}

Row 1 → {30, 40}

Row \ Col	0	1
0	10	20
1	30	40

for(int r=0; r<arr.length; r++) {

- Outer for loop for rows.
 - `arr.length = 2` → number of rows in `arr`.
 - `r` will take values 0 and 1.
 -
-

for(int c=0; c<arr.length; c++) {

- Inner for loop for columns.
-  **Note:** You are using `arr.length` here too. Since `arr` is 2x2, it works fine.
But in general, you should use `arr[r].length` (number of columns in that row).

So:

- When `r=0`, `c` takes values 0 and 1.
 - When `r=1`, `c` takes values 0 and 1.
 -
-

System.out.print(arr[r][c] + " ");

- Prints the element at **row `r` and column `c`**.
- `" "` adds space after each number (to separate them).
- Example:
 - When `r=0, c=0` → prints 10
 - When `r=0, c=1` → prints 20
 - When `r=1, c=0` → prints 30
 - When `r=1, c=1` → prints 40

System.out.println();

- Moves the cursor to the next line after printing each row.
- So row 0 elements appear in first line, row 1 elements in second line.

Output

10 20

30 40

 Copy code

java

 Copy code

```
package BasicConcept;

public class array2demo {
    public static void main(String[] args) {
        String arr[][]= {
            {"id", " Name", " Surname"},
            {"1", " himanshu", "Raghort"},
            {"2", " Shubham", " Kadam"}
        };

        for(int r=0; r<arr.length; r++) {           // loop rows
            for(int c=0; c<arr[r].length; c++) {      // loop columns
                System.out.print(arr[r][c] + " ");
            }
            System.out.println();
        }
    }
}
```

Your Code with Explanation

java

 Copy code

```
package BasicConcept;
```

1. Declares the package name → BasicConcept.
2. Organizes your class inside this package.

java

 Copy code

```
public class array2demo {
```

1. Defines a **public class** called array2demo.
2. All code runs inside this class.

java

 Copy code

```
public static void main(String[] args) {  
    // TODO Auto-generated method stub
```

1. main() → Entry point of Java program.
2. public → accessible from anywhere.
3. static → JVM calls it without creating object.
4. void → no return value.
5. String[] args → holds command-line arguments (not used here).

java

 Copy code

```
String arr[][] = {  
    {"id", " Name", " Surname"},  
    {"1", " himanshu", "Raghorte"},  
    {"2", " Shubham", " Kadam"}  
};
```

1. Declares a **2D String array** named arr.
2. Structure looks like:

id	Name	Surname
1	himanshu	Raghortte
2	Shubham	Kadam

- Internally in memory:

mathematica

 Copy code

```
Row 0 → {"id", " Name", " Surname"}
Row 1 → {"1", " himanshu", "Raghortte"}
Row 2 → {"2", " Shubham", " Kadam"}
```

java

 Copy code

```
for(int r=0; r<arr.length; r++) {
```

- Outer loop → iterates rows.
- `arr.length = 3` (3 rows).
- So `r = 0, 1, 2`.

java

 Copy code

```
for(int c=0; c<arr.length; c++) {
```

1. Inner loop → iterates **columns**.

⚠ **Issue here:** you used `arr.length` (which is rows).

👉 It works only for square arrays (same number of rows & columns).

2. But here, row length = 3 and column length = 3, so it luckily works.

3. Correct way:

java

Copy code

```
for(int c=0; c<arr[r].length; c++)
```

java

Copy code

```
System.out.print(arr[r][c]+" ");
```

- Prints the element at row `r` and column `c`.
- Adds a space after each element in the row.

java

Copy code

```
System.out.println("");
```

- Moves to next line after printing one full row.
- Ensures table-like display.

✅ Program Output

bash

Copy code

id	Name	Surname
1	himanshu	Raghorite
2	Shubham	Kadam



3D Array

```
package BasicConcept;  
  
public class MultidimensionalArray {  
  
    public static void main(String[] args) {  
        int a[][][] = {  
            { {1,2}, {3,4} },  
            { {5,6}, {7,8} }  
        };  
  
        for(int i=0; i<a.length; i++) {  
            for(int j=0; j<a.length; j++) {  
                for(int k=0; k<a.length; k++) {  
                    System.out.print(a[i][j][k] + " ");  
                }  
                System.out.println();  
            }  
            System.out.println();  
        }  
    }  
}
```

Step 1: Array Declaration

```
java  
  
int a[][][] = {  
    { {1,2}, {3,4} },  
    { {5,6}, {7,8} }  
};
```

1. a is a **3D array**.
2. It can be thought of as a **cube** → [blocks][rows][columns].

Structure:

mathematica

 Copy code

Block 0:

Row 0 → {1, 2}

Row 1 → {3, 4}

Block 1:

Row 0 → {5, 6}

Row 1 → {7, 8}

Step 2: Outer Loop

java

 Copy code

```
for(int i=0; i<a.length; i++)
```

1. $i \rightarrow$ selects the block.
2. $a.length = 2 \rightarrow$ 2 blocks (0 and 1).

Step 3: Middle Loop

java

 Copy code

```
for(int j=0; j<a.length; j++)
```

1. $j \rightarrow$ selects the row inside each block.
2. ⚠ Here you used `a.length` again (which is 2).
That works in this case (since both blocks have 2 rows),
but the safer way is:

java

 Copy code

```
for(int j=0; j<a[i].length; j++)
```

Step 4: Inner Loop

java

 Copy code

```
for(int k=0; k<a.length; k++)
```

1. $k \rightarrow$ selects the column inside each row.
2. ⚠ Again, you used `a.length`.
Correct version should be:

java

 Copy code

```
for(int k=0; k<a[i][j].length; k++)
```

Step 5: Print Statement

java

 Copy code

```
System.out.print(a[i][j][k] + " ");
```

Prints the element at block i, row j, column k

Step 6: Line Breaks

`System.out.println();`

- Moves to new line after each row.
-

`System.out.println();`

- Blank line after each block.

 Output

 Copy code

1 2

3 4

5 6

7 8

NEW CODE

```
package BasicConcept;

public class MultidimensionalArray {

    public static void main(String[] args) {
        int a[][][] = {
            { {1,2}, {3,4} },
            { {5,6}, {7,8} }
        };

        for(int i=0; i<a.length; i++) {                // blocks
            for(int j=0; j<a[i].length; j++) {          // rows
                for(int k=0; k<a[i][j].length; k++) {   // columns
                    System.out.print(a[i][j][k] + " ");
                }
                System.out.println();
            }
            System.out.println();
        }
    }
}
```